

2018-2019



Conduite de Projets Objet

Java
CTD 3

Anne-Gwenn Bosser

Plan du cours aujourd'hui – Objectifs pédagogiques

■ Sujets abordés aujourd'hui:

- retour sur des exercices importants du labo: Hulk, Chi Fu Mi
- Les petites choses: cast, égalité des objets, exceptions
- Bibliothèques: itérer sur des Collections, Streams

■ Plan des activités:

- Vous corrigez au tableau les exercices (45 minutes / 1h au besoin c'est important)
- Cours Magistral: cast, égalités, exceptions (15 minutes)
- Exercices devinettes (5 minutes)
- Cours Magistral et demos: un premier tour des bibliothèques Java, javadoc (~20 minutes)

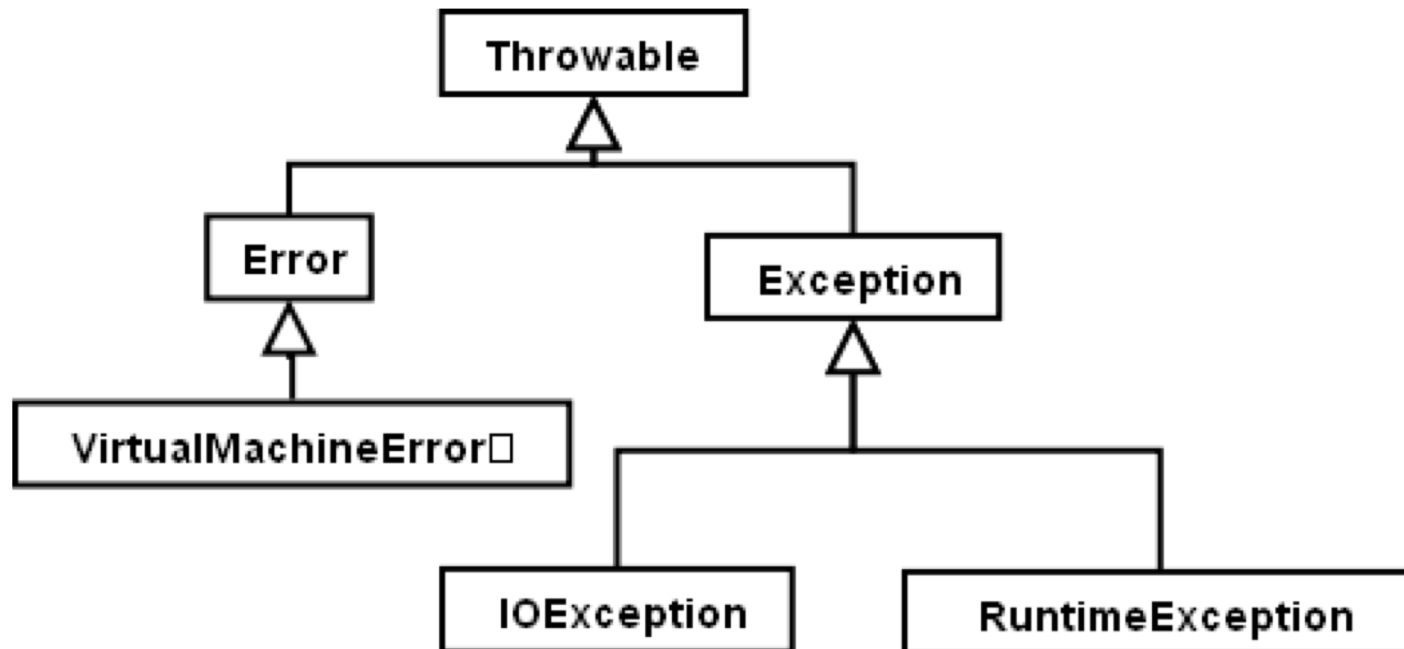
■ Après la séance vous devriez être capable de:

- ▶ recourir sans hésiter au polymorphisme objet et au polymorphisme paramétrique
- ▶ coder plusieurs manières de parcourir une collection.
- ▶ décrire les différentes sortes d'exceptions, savoir quand les rattrapper

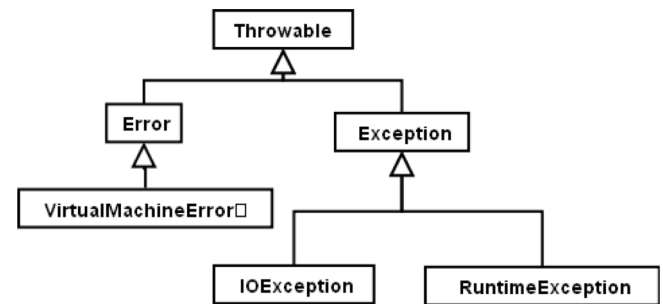
Exceptions

- Définir, et lancer une exception
- Rattraper ou propager une exception
- Les *RuntimeException* et les autres

- Manipulables au travers d'objets
- On peut en définir de nouvelles classes, étendant Throwable class.



Throwables



- **Error**: erreurs sévères, empêchant la machine virtuelle de fonctionner
 - Exemple: plus de mémoire disponible
 - Remarque: le développeur d'application ne devrait pas avoir besoin d'en définir de nouvelles
- **Exception**: moins grave (le programme s'arrête quand même...)
 - **IOException**: Erreurs d'entrées-sorties
 - Exemple. :EOFException (End of file Exception), FileNotFoundException,
 - **RuntimeException**: erreurs pouvant être générées durant une exécution normale :
 - Exemple. NullPointerException, ArrayOutOfBoundsException, ClassCastException

*Le/la développeur·se peut vouloir les gérer au niveau de l'application.
Le/la développeur·se sait qu'elles sont susceptibles de se produire.*

Vie d'une exception: Propagation

- Une exception Java est un objet, et peut donc être manipulée comme un objet.
- le mécanisme permettant la propagation de l'exception de la méthode la produisant à la méthode appelante est un autre flot de contrôle du programme.
- Une exception peut être:
 - **Levée/jetée** par une méthode: mot clé `throw` dans le corps de la méthode.
 - **Attrapée** dans une méthode: bloc `try/catch`
 - **Propagée** à la méthode appelante: mot clé `throws` dans le prototype de la méthode

```
public class MyHTTPException extends Exception{  
    public MyHTTPException(String message) {  
        super(message) ;  
    }  
}
```

```
public void testException() throws MyHTTPException{  
    throw new MyHTTPException("test exception") ;  
}
```

```
public void testPropagateException() throws MyHTTPException{  
    testException() ;  
}
```

```
public void testCatchException(){  
    try{  
        testPropagateException();  
    } catch (MyHTTPException myException){  
        System.out.println("exception caught: " + myException) ;  
    }  
}
```

exception caught: execeptionexample.MyHTTPException: test exception

Attraper ou Propager?

Quand une exception est levée

- Elle est propagée au dessus dans la pile d'appels (*call stack*)
- La propagation s'arrête si on se trouve dans un bloc `try/catch` qui correspond au type de l'exception
- Sinon, le programme s'arrête après avoir affiché les détails de l'exception et la pile d'appels interrompue.

```
public void testCatchException() {  
    try{  
  
        testPropagateException();  
  
    } catch (MyException e) {  
        System.out.println("exception MyException caught: " + e) ;  
  
    } catch (Exception e) {  
        System.out.println("exception Exception caught:" + e) ;  
  
    } finally{  
        System.out.println("finally is always executed") ;  
    }  
}
```

```
exception MyException  
caught:execeptionexample.MyHTTPException: test exception  
finally is always executed
```

- Le premier bloc **catch** dont le type correspond à l'exception est exécuté
- Le bloc **finally** est toujours exécuté.

Que se passe t-il ici?

```
public void testCatchException() {  
    try {  
        testPropagateException();  
    } catch (Exception e) {  
        System.out.println("exception Exception caught: " + e) ;  
    } catch (MyException e) {  
        System.out.println("exception MyException caught:" + e) ;  
    } finally {  
        System.out.println("finally is always executed") ;  
    }  
}
```

Que se passe t-il ici?

```
public void testCatchException() {  
    try {  
        testPropagateException();  
    } catch (Exception e) {  
        System.out.println("exception Exception caught: " + e);  
    } catch (MyException e) {  
        System.out.println("exception MyException caught:" + e);  
    } finally {  
        System.out.println("finally is always executed");  
    }  
}
```

Exception in thread "main" java.lang.Error: Unresolved compilation problem:
Unreachable catch block for MyHTTPException. It is already handled by the catch block for
Exception

at execeptionexample.MainProg.testCatchException_3_ (MainProg.java:65)
at execeptionexample.MainProg.main (MainProg.java:10)

Et ici?

```
public class TrickyExceptionTest {  
  
    static int h(int r){  
        System.out.println("h("+r+")") ;  
        return r ;  
    }  
    static void g(int i) throws Exception{  
        if (i==0)  
            throw new Exception() ;  
    }  
    static int f(){  
        try{  
            g(0) ;  
            return h(0) ;  
        }catch (Exception e){  
            System.out.println("Exception caught") ;  
            return h(1) ;  
        }finally{  
            System.out.println("finally") ;  
            return h(2) ;  
        }  
    }  
    public static void main(String[] args){  
        System.out.println("f returns: " + f()) ;  
    }  
}
```

Et ici?

```
public class TrickyExceptionTest {  
  
    static int h(int r){  
        System.out.println("h("+r+")");  
        return r;  
    }  
  
    static void g(int i) throws Exception{  
        if (i==0)  
            throw new Exception();  
    }  
  
    static int f(){  
        try{  
            g(0);  
            return h(0);  
        } catch (Exception e){  
            System.out.println("Exception caught");  
            return h(1);  
        } finally{  
            System.out.println("finally");  
            return h(2);  
        }  
    }  
  
    public static void main(String[] args){  
        System.out.println("f returns: " + f());  
    }  
}
```

```
Exception caught  
h(1)  
finally  
h(2)  
f returns: 2
```

Pourquoi utiliser des exceptions?

- Une manière élégante de résoudre les problèmes qui viennent d'ailleurs

- Ressources: input /output device, file, memory

- Interruptions de programmes quand les données ne conviennent pas

- Gestion d'erreur

- **Centralisée**

- (un bloc Try/Catch → plusieurs exceptions)

- **Explicite**

- On peut raffiner les erreurs en utilisant leur hiérarchie

- **Générique**

- (réutilisable)

- Plus d'informations

- [:http://java.sun.com/docs/books/tutorial/essential/exceptions/advantages.html](http://java.sun.com/docs/books/tutorial/essential/exceptions/advantages.html)

Checked / Unchecked Exception ?

Checked Exception

Doit être explicitement

Soit être gérée dans un bloc **Try/Catch**

Soit propagée au dessus

Unchecked Exception

Ne doit pas forcément être gérée explicitement.

Peuvent être définie seulement comme héritant de **RuntimeException**

Checked / Unchecked Exception example

```
public class A {  
  
    public void test()  
    {  
        throw new RuntimeException("RT example");  
        // do not need to be checked ( try/catch Method or propagated)  
    }  
  
    public void test2() throws IOException // wont compile if not explicitly propagated  
    {  
        throw new IOException("RT example");  
        // need to be checked  
    }  
}
```

Une méthode redéfinissant une autre méthode qui lance une exception peut seulement lancer une exception dont le type est compatible.

Même type ou sous-type

Conversion de type (cast)

Up and Down Casting

■ **Upcasting:** on peut remplacer une valeur d'un type par une autre d'un sous-type implicitement.

```
MySubClass refSC = new MySubClass();  
MyClass refC = refSC;  
Object refObj = refSC;
```

■ **Downcasting:** dans l'autre sens, il faut que cela soit explicite, et risque de lever une exception

```
MyClass refC = new MySubClass();  
MySubClass refSC = (MySubClass) refC;
```

Downcasting levant une RuntimeException

```
public static void main(String[] args) {
```

```
    A a = new A();
```

```
    B b = new B();
```

```
    B b2;
```

```
    try{
```

```
        b2 = (B) a;
```

```
    }catch(RuntimeException e){
```

```
        System.out.println(" \n Exception " + e);
```

```
    }
```

```
}
```

```
public class B extends A
```

```
<terminated> MainProg (9) [Java Application] C:\Program Files (x86)\Java\jre1.6.0_07\bin\javaw.exe (Oct 16, 2008 10:51
```

```
Exception java.lang.ClassCastException: test.A cannot be cast to test.B
```

```
public static void main(String[] args) {
```

```
    A a = new A();
```

```
    B b = new B();
```

```
    B b2;
```

```
    try{
```

```
        if(a instanceof B) {b2 = (B)a;
```

```
            System.out.println("CAST -1- DONE");}
```

```
        a=b;
```

```
        if(a instanceof B) {b2 = (B)a;
```

```
            System.out.println("CAST -2- DONE");}
```

```
    }catch(RuntimeException e){
```

```
        System.out.println(" \n Exception " + e);
```

```
    }
```

```
}
```

Operator "Instance of" cost is Less than a
Exception generation

```
CAST -2- DONE
```

■ From *Effective C++*, by Scott Meyers :

■ *"Anytime you find yourself writing code of the form:*

- *"if the object is of type T1, then do something, but if it's of type T2, then do something else",*
- ***slap yourself.** "*
- → use overriding/overloading (or Visitor Pattern)